

python.closure#

<https://pytorch.org/docs/stable/generated/exportdb/python.closure.html>

Description:

Tags: torch.cond, python.closure Support Level: SUPPORTED Original source code:

Code Examples

```

# mypy: allow-untyped-defs
import torch

from functorch.experimental.control_flow import cond

class CondClosedOverVariable(torch.nn.Module):
    """
    torch.cond() supports branches closed over arbitrary
    variables.
    """

    def forward(self, pred, x):
        def true_fn(val):
            return x * 2

        def false_fn(val):
            return x - 2

        return cond(pred, true_fn, false_fn, [x + 1])

example_args = (torch.tensor(True), torch.randn(3, 2))
tags = {"torch.cond", "python.closure"}
model = CondClosedOverVariable()

torch.export.export(model, example_args)

```

ExportedProgram:

```
class GraphModule(torch.nn.Module):
    def forward(self, pred: "b8[]", x: "f32[3, 2]"):
        add: "f32[3, 2]" = torch.ops.aten.add.Tensor(x,
1); add = None

        true_graph_0 = self.true_graph_0
        false_graph_0 = self.false_graph_0
        cond = torch.ops.higher_order.cond(pred,
true_graph_0, false_graph_0, (x,)); pred = true_graph_0 =
false_graph_0 = x = None
        getitem: "f32[3, 2]" = cond[0]; cond = None
        return (getitem,)

class true_graph_0(torch.nn.Module):
    def forward(self, x: "f32[3, 2]"):
        mul: "f32[3, 2]" =
torch.ops.aten.mul.Tensor(x, 2); x = None
        return (mul,)

class false_graph_0(torch.nn.Module):
    def forward(self, x: "f32[3, 2]"):
        sub: "f32[3, 2]" =
torch.ops.aten.sub.Tensor(x, 2); x = None
        return (sub,)
```

Graph signature:

```
# inputs
pred: USER_INPUT
x: USER_INPUT

# outputs
getitem: USER_OUTPUT
```

Range constraints: {}

```
# mypy: allow-untyped-defs
import torch

class NestedFunction(torch.nn.Module):
    """
    Nested functions are traced through. Side effects on global
    captures
    are not supported though.
    """

    def forward(self, a, b):
        x = a + b
        z = a - b

        def closure(y):
            nonlocal x
            x += 1
            return x * y + z

        return closure(x)

example_args = (torch.randn(3, 2), torch.randn(2))
tags = {"python.closure"}
model = NestedFunction()

torch.export.export(model, example_args)
```

ExportedProgram:

```
class GraphModule(torch.nn.Module):
    def forward(self, a: "f32[3, 2]", b: "f32[2]"):
        add: "f32[3, 2]" = torch.ops.aten.add.Tensor(a,
b)

        sub: "f32[3, 2]" = torch.ops.aten.sub.Tensor(a,
b); a = b = None

        add_: "f32[3, 2]" =
torch.ops.aten.add_.Tensor(add, 1); add = None

        mul: "f32[3, 2]" =
torch.ops.aten.mul.Tensor(add_, add_); add_ = None
        add_1: "f32[3, 2]" = torch.ops.aten.add.Tensor(mul,
sub); mul = sub = None
        return (add_1,)
```

Graph signature:

```
# inputs
a: USER_INPUT
b: USER_INPUT

# outputs
add_1: USER_OUTPUT
```

Range constraints: {}

Notes & Warnings

NOTE Note Tags: torch.cond, python.closure Support Level: SUPPORTED

NOTE Note Tags: python.closure Support Level: SUPPORTED

Detailed Documentation

cond_closed_over_variable#

Original source code:

nested_function#

Original source code:

Tags: torch.cond, python.closure

Support Level: SUPPORTED

Original source code:

Support Level: SUPPORTED

Original source code:

